# 03 · Plan the Work

> *Audience: project/program managers, planners · Time: 45 minutes*
> *Prerequisite: module 01 · Screens: Plan group (`#/register`, `#/wbs`, `#/tasks`, `#/schedule`, `#/milestones`, `#/baselines`, `#/tools`)*

## Learning objectives

Create a project from the register, decompose it into WBS and tasks, build a critical-path schedule with dependencies, lock a baseline, and use the Planning Lab to test "what-if" scenarios before committing.

---

## 1. The planning chain

```
Project (register) → WBS elements → Tasks (effort, owner, dates)
       → Dependencies (finish-to-start etc.) → CPM schedule (critical path)
       → Milestones → Baseline snapshot → continuous tracking
```

Each step feeds the next automatically. Nothing is re-keyed.

## 2. Project Register (`#/register`)

The operational twin of the Projects screen: one row per project with code, name, program, PM, stage, RAG, budget and dates.

- **Create** a project: code (e.g. `PRJ-AURORA`), name, program, PM, planned start/finish, budget.
- **Double-click any cell** to edit it in place — the fastest way to keep register data current.
- **CSV import**: use Import to bulk-load projects from a spreadsheet (the import is idempotent — running it twice does not duplicate). Export first to get the exact column template.
- Filters, search and column sorting work on every list screen in the platform.

## 3. Requirements (`#/requirements`)

Capture *what* must be delivered before planning *how*:

- Link each requirement to a project (and optionally to WBS elements).
- Track status (Draft → Approved → Delivered) and priority.
- Approved requirements become the scope boundary a change request is measured against — this is what makes scope creep visible.

## 4. WBS (`#/wbs`)

Decompose the project into a hierarchical tree:

1. Pick the project (picker at the top).
2. Add level-1 elements (phases, e.g. *Design → Build → Test → Deploy*).
3. Add children under each phase; drag to re-parent; the tree shows effort roll-ups automatically.
4. Any WBS element can own tasks — the tree is the skeleton, tasks are the muscle.

## 5. Tasks (`#/tasks`)

Every task carries: name, WBS parent, owner, planned effort (h), start/finish, % complete, status, priority.

- **Double-click** a cell to edit inline; open the drawer for the full form.
- **Auto-scheduling**: with a project calendar and dependencies in place, dates recalculate from the critical path. Without dependencies, tasks stay where you put them (relative-date mode).
- Tasks roll up: % complete feeds WBS and project health; actual effort feeds EVM; overdue tasks feed alerts and the automations engine.

## 6. Schedule & critical path (`#/schedule`)

The upgraded CPM (Critical Path Method) engine:

- **Gantt view** — bars per task; critical-path tasks highlighted; drag to move.
- **Network view** — the dependency network (CpmNetwork) with computed early start/finish, late dates and **total float**.
- **Constraints & deadlines** — ASAP by default; add "must-start-on", "finish-no-later-than" or deadlines when contracts demand them. The engine reports conflicts (negative float) instead of silently accepting them.
- **Calendar** — working days/exceptions drive every date computation.

> *Health check: the platform continuously verifies the plan — orphan tasks, deadline breaches and unassigned effort are flagged by the plan doctor (Planning Lab). A healthy connected plan scores 100.*

## 7. Milestones (`#/milestones`)

Zero-duration markers (gate reviews, go-live, handover). Missed milestones:
- appear on the Executive dashboard (overdue milestones tile),
- fire the `MILESTONE_MISSED` automation trigger.

## 8. Baselines (`#/baselines`)

When the plan is approved, **snapshot it**:

1. Open Baselines, click New, name it (e.g. "Baseline v1 — board approved").
2. The baseline freezes scope, dates and cost at that moment.
3. All variance reporting (schedule/cost deviations on EVM and Health screens) is measured against the selected baseline — this is the contract with the sponsor.

Never edit a baseline after approval — create a new one through change control.

## 9. Planning Lab (`#/tools`) — the PM's flight simulator

Four instruments:

1. **CPM calculator** — run critical-path math on the live plan; inspect ES/EF/LS/LF and float per task; see exactly which chain drives the finish date.
2. **What-if scenarios** — model "what if we add two engineers" or "what if the vendor slips two weeks" without touching the live plan. Compare scenario vs baseline side by side, then adopt the winner.
3. **Resource leveling** — detect over-allocations on working days and generate *prescriptions* (suggested moves). The engine never touches critical tasks and honours float — prescriptions are proposals, you stay in control.
4. **Plan doctor** — a full diagnostic: orphans, deadline breaches, unassigned effort, calendar gaps. Run it before every gate review.

> *The seeded demo project **PRJ-CZR-FLOW** is a pristine reference plan: 364 tasks, 44 critical, 2026-10-19 → 2028-04-01, FLW-001 with 521 days of float. Use it to demonstrate what a healthy plan looks like.*

## 10. Exercise (20 min) — plan a mini-project

1. Create project `PRJ-TRAINING` in the register (start next Monday, 4 weeks).
2. In WBS, add phases: *Initiate, Design, Build, Test, Close*.
3. Add 2–3 tasks per phase with owners and effort; double-click to fix a date.
4. In Schedule, link Design → Build → Test with finish-to-start dependencies.
5. Open the **Network view** — identify the critical path and total float.
6. Run the **Plan doctor** — fix everything it flags; aim for score 100.
7. Create **Baseline v1**.

**Discussion**: which task, if delayed one week, pushes the whole project? How long is your float?

## 11. Key takeaways

- Plan once, reuse everywhere: tasks feed schedule, EVM, health, alerts and reports — no double bookkeeping.
- Dependencies + calendar = critical path; the Planning Lab makes it testable.

- Baselines turn "we're late" into a measurable, governed variance.